



TodoTec v2

Sistema ERP + Tienda Online

Documentación Técnica Completa

Stack	FastAPI · React · Next.js · SQLite/PostgreSQL
Puertos	Backend :8000 · Admin :5173 · Tienda :3000
Versión	2.0.0 — Junio 2026
Autor	TodoTec — Jose Rojas

Tabla de Contenidos

- 1.** Resumen del Sistema
- 2.** Arquitectura General
- 3.** Stack Tecnológico
- 4.** Estructura del Proyecto
- 5.** Backend — FastAPI
 - 5.1 Modelos de datos (ORM)
 - 5.2 Esquemas Pydantic
 - 5.3 Referencia de API endpoints
 - 5.4 Lógica de negocio clave
- 6.** Admin Dashboard — React
 - 6.1 Estructura de componentes
 - 6.2 Módulos y páginas
- 7.** Tienda Online — Next.js
 - 7.1 Páginas y rutas
 - 7.2 Sincronización con backend
- 8.** Base de Datos — Esquema completo
- 9.** Flujos de negocio
- 10.** Configuración e instalación (Fase 1)
- 11.** Migración a producción (Fase 2)
- 12.** Guía para desarrolladores

1. Resumen del Sistema

TodoTec v2 es una plataforma integrada de gestión empresarial (ERP) y comercio electrónico diseñada para una empresa de importación y comercialización de productos tecnológicos. El sistema permite gestionar el ciclo completo del negocio: desde la compra e importación de productos hasta la venta, control de inventario, generación de estados financieros y comercialización a través de una tienda online.

Módulo	Descripción
Gestión de Proveedores	CRUD completo con rating, búsqueda y filtros
Compras e Inventario	Registro de compras con variantes de color, SKU automático, imágenes por color
Ventas	Carrito multi-producto, comprobante imprimible, ventas retail y mayorista
Contabilidad	P&L, Balance General, aportes de capital, asientos automáticos, reset
Categorías	CRUD de categorías sincronizadas con la tienda online
Configuración	Datos empresa, IGV, serie de facturación, redes sociales
Tienda Online	Catálogo, búsqueda, filtros, detalle con imagen por color, carrito, WhatsApp
Dashboard	KPIs de ventas, inventario, rentabilidad, gráficos de tendencia

2. Arquitectura General

El sistema sigue una arquitectura de tres capas desacopladas que se comunican exclusivamente a través de la API REST del backend. Ningún frontend accede directamente a la base de datos.

Capa	Tecnología	Puerto	Descripción
Backend	FastAPI + Python	:8000	API REST, lógica de negocio, acceso a BD
Admin Dashboard	React 18 + Vite	:5173	Panel de gestión interno (ERP)
Tienda Online	Next.js 14	:3000	E-commerce público con SSR
Base de Datos	SQLite → PostgreSQL	archivo local	Persistencia de todos los datos
Archivos	Sistema de archivos local	/uploads	Imágenes de productos y logo

Principio de comunicación:

Admin (React) → HTTP/JSON → FastAPI → SQLAlchemy ORM → SQLite
 Tienda (Next.js) → HTTP/JSON → FastAPI → SQLAlchemy ORM → SQLite
 Admin y Tienda NUNCA se comunican entre sí directamente.

3. Stack Tecnológico

Categoría	Tecnología	Versión	Propósito
Backend framework	FastAPI	0.111	API REST, validación automática, docs Swagger
ORM	SQLAlchemy	2.0	Mapeo objeto-relacional, queries
Validación	Pydantic v2	2.7	Schemas de entrada/salida, serialización
Servidor ASGI	Uvicorn	0.29	Servidor HTTP async para FastAPI
BD Fase 1	SQLite	built-in	Base de datos local sin instalación
BD Fase 2	PostgreSQL	15+	Base de datos en producción (Supabase)
Admin framework	React 18	18.3	UI del panel de gestión
Admin builder	Vite	5.2	Bundler ultra-rápido con HMR
Admin estilos	Tailwind CSS	3.4	Utilidades CSS, diseño responsive
Tienda framework	Next.js	14.2	SSR/SSG para SEO, rutas automáticas
Gráficos admin	Recharts	2.12	Gráficos para dashboard
Notificaciones	react-hot-toast	2.4	Toasts de confirmación/error
HTTP client	Axios	1.7	Llamadas HTTP desde admin
Iconos	Lucide React	0.383	Iconografía consistente

4. Estructura del Proyecto

Árbol de directorios principal:

```

todotec_v2/
├── backend/                                # FastAPI application
│   └── app/
│       ├── main.py                        # Entry point, CORS, rutas
│       ├── database.py                    # Engine SQLAlchemy, sesión
│       ├── models.py                      # Modelos ORM (tablas)
│       ├── schemas.py                    # Schemas Pydantic
│       └── routers/                       # Un archivo por módulo
│           ├── config.py                  # /api/config
│           ├── suppliers.py               # /api/suppliers
│           ├── products.py                # /api/products
│           ├── purchases.py               # /api/purchases
│           ├── sales.py                   # /api/sales
│           ├── accounting.py              # /api/accounting
│           ├── categories.py              # /api/categories
│           └── dashboard.py               # /api/dashboard
│       ├── uploads/                       # Imágenes subidas
│       ├── todotec.db                     # Base de datos SQLite
│       └── requirements.txt
│
├── admin/                                  # React + Vite (ERP)
│   └── src/
│       ├── App.jsx                        # Router principal
│       ├── api/client.js                  # Todas las llamadas API
│       ├── components/
│       │   ├── Layout.jsx                 # Sidebar + topbar
│       │   └── pages/
│       │       ├── Dashboard.jsx
│       │       ├── Proveedores.jsx
│       │       ├── Inventario.jsx
│       │       ├── Compras.jsx
│       │       ├── Ventas.jsx
│       │       ├── Contabilidad.jsx
│       │       ├── Categorías.jsx
│       │       └── Configuración.jsx
│       └──
│
└── store/                                  # Next.js (tienda pública)
    └── app/
        ├── layout.js                      # Layout global + navbar
        ├── page.js                        # Home page
        ├── catalog/page.js                # Catálogo con filtros
        ├── product/[id]/                 # Detalle de producto
        └── cart/page.js                    # Carrito de compras
    
```

```

■ ■ ■■■ contact/page.js      # Página de contacto
■ ■■■ components/
■ ■ ■■■ Navbar.js           # Búsqueda + categorías
■ ■ ■■■ Footer.js
■ ■ ■■■ ProductCard.js
■ ■■■ lib/api.js            # Helpers fetch API
■
■■■ logo.svg
■■■ Iniciar TodoTec.command  # Launcher Mac

```

5. Backend — FastAPI

5.1 Modelos de datos (ORM)

Definidos en **backend/app/models.py** usando SQLAlchemy ORM. Cada clase corresponde a una tabla en la base de datos.

CompanyConfig

Configuración global de la empresa. Solo existe una fila (id=1). Almacena nombre comercial, RUC, IGV, serie de facturación, redes sociales y logo.

Supplier

Proveedores del negocio. Contiene nombre, email, teléfono, ciudad y rating. Relacionado con Purchase.

Product

Producto del catálogo. Tiene SKU único generado automáticamente (TT-XXXXXXX). Incluye propiedades calculadas `total_stock` y `unit_cost` (costo promedio ponderado).

ProductVariant

Variante de color de un producto. Cada color tiene su propio stock e `image_url`. Es la unidad mínima de inventario.

ProductImage

Imágenes generales del producto (galería). Una puede ser `is_primary=True`.

Purchase

Cabecera de una compra. Relaciona proveedor, fecha, costos de envío, impuestos y si es a crédito.

PurchaseItem

Línea de una compra: qué producto, cuántas unidades, a qué costo, con qué variantes (guardado como JSON en `variants_data`).

Sale

Cabecera de una venta. Incluye datos del cliente, total, IGV, número de comprobante y tipo (retail/mayorista).

SaleItem

Línea de una venta: producto, variante específica, cantidad, precio catálogo y precio real de venta.

AccountingEntry

Asiento contable. Generado automáticamente por cada compra, venta o aporte de capital. Contiene cuenta débito, cuenta crédito y monto.

CapitalContribution

Aporte de capital por un socio. Genera un asiento Efectivo / Capital Social automáticamente.

Category

Categoría estandarizada de productos. Tiene nombre único, descripción, orden y visibilidad. Precreadas al primer uso.

5.2 Schemas Pydantic

Definidos en **backend/app/schemas.py**. Cada modelo tiene tres schemas: **Create** (datos de entrada), **Update** (actualización parcial) y **Out** (respuesta serializada). Los schemas Out incluyen campos calculados como `total_stock` y `unit_cost`.

5.3 Referencia de API Endpoints

Todos los endpoints están documentados automáticamente en <http://localhost:8000/docs> (Swagger UI) y <http://localhost:8000/redoc>. A continuación la referencia completa por módulo.

Configuración (/api/config)

Endpoint	Método	Descripción	Notas
GET /api/config	GET	Obtener configuración actual	Crea una fila por defecto si no existe
PUT /api/config	PUT	Actualizar configuración	Actualización parcial con campos opcionales
POST /api/config/logo	POST	Subir logo de empresa	Multipart/form-data, guarda en /uploads

Proveedores (/api/suppliers)

Endpoint	Método	Descripción	Notas
GET /api/suppliers	GET	Listar proveedores	Param: ?search=texto
GET /api/suppliers/{id}	GET	Obtener un proveedor	404 si no existe
POST /api/suppliers	POST	Crear proveedor	Status 201
PUT /api/suppliers/{id}	PUT	Actualizar proveedor	Actualización parcial
DELETE /api/suppliers/{id}	DELETE	Eliminar proveedor	—

Productos (/api/products)

Endpoint	Método	Descripción	Notas
GET /api/products	GET	Listar productos	Params: search, category, store_only, active_only
GET /api/products/categories	GET	Categorías usadas en productos	Valores distintos de product.category
GET /api/products/{id}	GET	Obtener producto con variantes e imágenes	—
POST /api/products	POST	Crear producto	SKU auto-generado TT-XXXXXXX
PUT /api/products/{id}	PUT	Actualizar producto	Actualización parcial
DELETE /api/products/{id}	DELETE	Eliminar producto	Cascada: variantes e imágenes
POST /api/products/{id}/images	POST	Subir imagen al producto	Primera imagen es is_primary=True
DELETE /api/products/images/{id}	DELETE	Eliminar imagen	Borra archivo físico
POST /api/products/{id}/variants	POST	Agregar variante de color	—
PUT /api/products/variants/{id}	PUT	Actualizar variante	—
DELETE /api/products/variants/{id}	DELETE	Eliminar variante	—
POST /api/products/variants/{id}/image	POST	Subir foto de un color	Reemplaza foto anterior
DELETE /api/products/variants/{id}/image	DELETE	Eliminar foto de un color	—

Compras (/api/purchases)

Endpoint	Método	Descripción	Notas
GET /api/purchases	GET	Listar compras	Ordenadas por fecha desc

Endpoint	Método	Descripción	Notas
GET /api/purchases/{id}	GET	Detalle de una compra	Incluye items y proveedor
POST /api/purchases	POST	Registrar compra	Valida efectivo si is_credit=False. Crea productos/variantes
DELETE /api/purchases/{id}	DELETE	Eliminar compra	Revierte el stock de cada variante

Ventas (/api/sales)

Endpoint	Método	Descripción	Notas
GET /api/sales	GET	Listar ventas	Ordenadas por fecha desc
GET /api/sales/{id}	GET	Detalle de una venta	Incluye items con producto y variante
POST /api/sales	POST	Registrar venta	Valida stock por variante. Reduce stock. Genera asiento
DELETE /api/sales/{id}	DELETE	Eliminar venta	Revierte el stock de cada variante

Contabilidad (/api/accounting)

Endpoint	Método	Descripción	Notas
GET /api/accounting/entries	GET	Listar todos los asientos	Orden: fecha desc
GET /api/accounting/cash	GET	Efectivo disponible	suma(débito Efectivo) - suma(crédito Efectivo)
GET /api/accounting/income-statement	GET	Estado P&L	Ventas, COGS, utilidad bruta y neta
GET /api/accounting/balance-sheet	GET	Balance General	Activos, pasivos, patrimonio
GET /api/accounting/capital	GET	Aportes de capital	—
POST /api/accounting/capital	POST	Registrar aporte	Genera asiento Efectivo/Capital Social
DELETE /api/accounting/capital/{id}	DELETE	Eliminar aporte	—
POST /api/accounting/reset	POST	Reset balance	Elimina TODOS los asientos y aportes. Reinicia correlativo

Categorías (/api/categories)

Endpoint	Método	Descripción	Notas
GET /api/categories	GET	Categorías activas	Auto-pobla con 8 categorías por defecto si está vacío
GET /api/categories/all	GET	Todas las categorías	Incluye inactivas — para el admin
POST /api/categories	POST	Crear categoría	400 si ya existe el nombre
PUT /api/categories/{id}	PUT	Actualizar categoría	Puede cambiar is_active para ocultar/mostrar
DELETE /api/categories/{id}	DELETE	Eliminar categoría	—

Dashboard (/api/dashboard)

Endpoint	Método	Descripción	Notas
GET /api/dashboard	GET	KPIs ejecutivos completos	Ventas hoy/semana/mes/año, inventario, top productos, top clientes

Reset Total (/api/reset-total)

Endpoint	Método	Descripción	Notas
POST /api/reset-total	POST	Eliminar TODOS los registros operacionales	Mantiene config y categorías. Borra productos, compras,

5.4 Lógica de negocio clave

Flujo de Compra

1. El admin envía POST /api/purchases con items, variantes, costos de envío e impuestos.
2. Si is_credit=False: se verifica que efectivo disponible >= costo total estimado.
Si no hay suficiente efectivo: HTTP 400 con mensaje descriptivo.
3. Para cada item: si product_id=None, se crea el producto con SKU auto-generado.
4. El costo extra (envío + impuestos) se distribuye proporcionalmente entre todos los items.
5. Para cada variante de color: si ya existe (mismo product_id + color) se suma el stock, si no existe se crea una nueva ProductVariant.
6. Si no hay variantes definidas, el stock va a una variante "Estándar" automática.
7. Se crea el asiento contable:
 - Al contado: Débito Inventarios / Crédito Efectivo
 - A crédito: Débito Inventarios / Crédito Cuentas por Pagar

Flujo de Venta

1. El admin envía POST /api/sales con items (producto, variante, cantidad, precio).
2. Para cada item se verifica stock suficiente en la variante específica.
3. Se calcula subtotal, IGV (18% configurable) y total.
4. Se genera el número de comprobante: {serie}-{correlativo} (ej: F001-00000001).
5. El correlativo se incrementa en 1 en CompanyConfig.
6. Se reduce el stock de cada variante vendida.
7. Se crean dos asientos contables:
 - Venta: Débito Efectivo / Crédito Ventas (o CxC si is_credit=True)
 - COGS: Débito Costo de Ventas / Crédito Inventarios

Cálculo de Efectivo Disponible

```
efectivo = SUM(accounting_entries WHERE debit_account="Efectivo") - SUM(accounting_entries WHERE credit_account="Efectivo")
```

Este valor se usa en el Balance General y como validación antes de registrar una compra al contado.

Costo Promedio Ponderado (CPP)

El costo unitario de un producto se calcula como el promedio ponderado de todos los purchase_items del producto: $\text{SUM}(\text{quantity} * \text{unit_cost}) / \text{SUM}(\text{quantity})$. Este valor se usa en el asiento COGS al momento de vender.

6. Admin Dashboard — React

6.1 Estructura de componentes

El admin es una Single Page Application (SPA) con React Router v6. El enrutamiento está definido en **App.jsx**. El layout global (sidebar + topbar) está en **components/Layout.jsx** y se aplica a todas las páginas via **<Outlet />**.

6.2 Módulos y páginas

Página	Ruta	Descripción
Dashboard	/	KPIs de ventas (hoy/semana/mes/año), gráficos de tendencia y categorías, top productos, al
Proveedores	/proveedores	Tabla con búsqueda. Modal de creación/edición. Rating con estrellas. Eliminación con confir
Inventario	/inventario	Grid de tarjetas de productos. Muestra stock por color, margen, imágenes. Modal de edición
Compras	/compras	Lista de compras expandibles. Formulario de nueva compra con múltiples productos, variant
Ventas	/ventas	Lista de ventas expandibles. Formulario de nueva venta con carrito (selector de producto + c
Contabilidad	/contabilidad	4 pestañas: P&L, Balance General (con indicador de efectivo y botón reset), Aportes de Cap
Categorías	/categorias	Tabla CRUD de categorías. Permite crear, editar nombre/descripción/orden, activar/desactiv
Configuración	/configuracion	Datos empresa, logo, parámetros financieros (IGV, tipo de cambio), serie de facturación, red

Comunicación con el backend (api/client.js)

Todas las llamadas API están centralizadas en **src/api/client.js**. Se usa Axios con `baseUrl="/api"` y el proxy de Vite redirige al backend en `:8000`. Cada función exportada corresponde a un endpoint.

7. Tienda Online — Next.js

7.1 Páginas y rutas

Página	Ruta	Tipo	Descripción
Home	/	Server	Hero banner, barra de confianza, Top Sellers (8 slots), CTA mayorista
Catálogo	/catalog	Server	Grid de productos. Pills de categoría. Parámetros: ?search, ?category
Detalle	/product/[id]	Server+Client	Galería con imagen reactiva por color. Selector de variante. Agregar al carrito.
Carrito	/cart	Client	Lista de items, ajuste de cantidades, totales con IGV, pedido por WhatsApp
Contacto	/contact	Client	Info de contacto. Formulario que abre WhatsApp con el mensaje pre-llenado.

7.2 Sincronización con backend

La tienda consume el mismo backend que el admin. Los datos se actualizan en tiempo real: cualquier cambio en el admin (nuevo producto, precio actualizado, foto subida) se refleja inmediatamente en la tienda al recargar la página.

Dato	Endpoint	Notas
Catálogo	/api/products?store_only=true	Solo productos con show_in_store=True e is_active=True
Categorías Navbar	/api/categories	Cargadas dinámicamente con useEffect en el Navbar
Categorías filtros	/api/categories	Misma fuente — cambia en admin, refleja en filtros
Detalle producto	/api/products/{id}	Incluye variantes con stock y image_url por color
Imágenes	/uploads/{filename}	Servidas como archivos estáticos por FastAPI

Nota: el carrito se guarda en sessionStorage del navegador (no en el servidor). Al cerrar el navegador se pierde. Los pedidos se realizan a través de WhatsApp (+51 904 811 639).

8. Base de Datos — Esquema completo

SQLite en Fase 1 (archivo **backend/todotec.db**). Compatible con PostgreSQL sin cambiar los modelos — solo cambiar DATABASE_URL en database.py.

products — columnas	Tipo	Restricción	Descripción
id	INTEGER	PK, autoincrement	Identificador único
sku	VARCHAR	UNIQUE, NOT NULL	Código SKU (TT-XXXXXXX)
name	VARCHAR	NOT NULL	Nombre del producto
category	VARCHAR	—	Nombre de la categoría
description	TEXT	—	Descripción larga
sale_price	FLOAT	default 0	Precio de venta en S/
is_active	BOOLEAN	default True	Si el producto está activo
show_in_store	BOOLEAN	default True	Si aparece en la tienda online
created_at	DATETIME	server_default	Fecha de creación

product_variants — columnas	Tipo	Restricción	Descripción
id	INTEGER	PK	Identificador único
product_id	INTEGER	FK products.id	Producto al que pertenece
color	VARCHAR	NOT NULL	Nombre del color (ej: "Negro")
stock	INTEGER	default 0	Unidades disponibles de este color
image_url	VARCHAR	default ""	URL de la foto de este color (/uploads/...)

purchases / purchase_items — columnas	Tipo	Restricción	Descripción
id	INTEGER	PK	Identificador único
supplier_id	INTEGER	FK suppliers.id	Proveedor (opcional)
purchase_date	DATETIME	server_default	Fecha de la compra
shipping_cost	FLOAT	default 0	Costo de envío total
taxes	FLOAT	default 0	Impuestos de importación
total_cost	FLOAT	—	Costo total calculado
is_credit	BOOLEAN	default False	Si es compra a crédito
— items —	—	—	—
purchase_id	INTEGER	FK purchases.id	Compra a la que pertenece
product_id	INTEGER	FK products.id	Producto comprado
quantity	INTEGER	NOT NULL	Unidades compradas
unit_cost	FLOAT	NOT NULL	Costo unitario (con extras proporcionales)
variants_data	JSON	—	Lista [{color, qty}] de variantes en esta línea

accounting_entries — columnas	Tipo	Restricción	Descripción
id	INTEGER	PK	Identificador único
entry_date	DATETIME	server_default	Fecha del asiento

accounting_entries — column	tipo	Restricción	Descripción
entry_type	VARCHAR	NOT NULL	"compra", "venta", "cogs", "capital"
description	VARCHAR	NOT NULL	Texto descriptivo del asiento
debit_account	VARCHAR	NOT NULL	Cuenta debitada
credit_account	VARCHAR	NOT NULL	Cuenta acreditada
amount	FLOAT	NOT NULL	Monto del asiento en S/
purchase_id	INTEGER	FK purchases.id	Compra relacionada (opcional)
sale_id	INTEGER	FK sales.id	Venta relacionada (opcional)
capital_id	INTEGER	FK capital_contributions.id	Aporte relacionado (opcional)

9. Flujos de Negocio

Tabla de asientos contables automáticos

Operación	Cuenta Débito	Cuenta Crédito	Monto
Compra al contado	Inventarios	Efectivo	Total costo
Compra a crédito	Inventarios	Cuentas por Pagar	Total costo
Venta al contado	Efectivo	Ventas	Total venta (con IGV)
Venta a crédito	Cuentas por Cobrar	Ventas	Total venta (con IGV)
COGS (costo de venta)	Costo de Ventas	Inventarios	Costo promedio ponderado × qty
Aporte de capital	Efectivo	Capital Social	Monto del aporte

10. Configuración e Instalación — Fase 1 (Local)

Requisitos previos:

- Python 3.10 o superior — python.org
- Node.js 18 o superior — nodejs.org

Opción A — Script automático (recomendado para Mac):

doble clic en: "Iniciar TodoTec.command"

El script instala todas las dependencias la primera vez y levanta los 3 servicios automáticamente.

Opción B — Manual (3 terminales):

Backend

```
# Terminal 1 – Backend
cd todotec_v2/backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

Admin Dashboard

```
# Terminal 2 – Admin
cd todotec_v2/admin
npm install
npm run dev
```

Tienda Online

```
# Terminal 3 – Tienda
cd todotec_v2/store
npm install
npm run dev
```

URLs locales:

Servicio	URL
Admin Dashboard	http://localhost:5173
Tienda Online	http://localhost:3000
Backend API	http://localhost:8000
Swagger Docs	http://localhost:8000/docs
ReDoc	http://localhost:8000/redoc

11. Migración a Producción — Fase 2

El stack está diseñado para escalar sin refactor. Los únicos cambios necesarios para pasar a producción son variables de entorno.

Servicios recomendados:

Servicio	Proveedor	Costo	Notas
Base de datos	Supabase (PostgreSQL)	\$0 → \$25/mes	Tier free: 500MB
Backend hosting	Railway	\$5/mes	Deploy desde GitHub, SSL incluido
Admin hosting	Vercel	\$0	Tier hobby gratuito, deploy automático
Tienda hosting	Vercel	\$0	Next.js optimizado, CDN global
Imágenes	Cloudinary	\$0 → \$25/mes	Tier free: 25GB, transformaciones
Emails	Resend	\$0	100 emails/día gratis
Pagos	Culqi + PayPal	2.9% + S/0.10	Culqi primario, PayPal respaldo
Dominio	.com.pe	\$15/año	En NIC.pe o registrador local

Cambios de código para Fase 2:

Migración de base de datos

```
# backend/app/database.py
# Cambiar SOLO esta línea:
DATABASE_URL = "sqlite:///./todotec.db"           # Fase 1
DATABASE_URL = os.getenv("DATABASE_URL")         # Fase 2 (PostgreSQL)
```

CORS producción

```
# backend/app/main.py – CORS origins
allow_origins=["https://admin.todotec.pe", "https://www.todotec.pe"]
```

Variables de entorno

```
# Variables de entorno (.env en Railway)
DATABASE_URL=postgresql://user:pass@host:5432/todotec
ALLOWED_ORIGINS=https://admin.todotec.pe,https://www.todotec.pe

# Variables en Vercel (admin + store)
NEXT_PUBLIC_API_URL=https://api.todotec.pe
VITE_API_URL=https://api.todotec.pe
```

12. Guía para Desarrolladores

Agregar un nuevo módulo al backend

1. Crear el modelo en models.py (clase que hereda de Base)
2. Agregar el schema en schemas.py (Create, Update, Out)
3. Crear el router en routers/nuevo_modulo.py
4. Importar y registrar en main.py: app.include_router(nuevo_modulo.router)
5. Ejecutar el backend — SQLAlchemy crea la tabla automáticamente en el siguiente arranque

Agregar una página al admin

1. Crear src/pages/NuevaPagina.jsx
2. Agregar la ruta en App.jsx: } />
3. Agregar el ítem de nav en components/Layout.jsx (array NAV)
4. Si necesita llamadas API, agregar las funciones en api/client.js

Convenciones del proyecto

Backend:

- Nombres de modelos: PascalCase (Product, SaleItem)
- Nombres de columnas: snake_case (sale_price, is_active)
- Endpoints REST estándar: GET lista, GET /{id} detalle, POST crea, PUT actualiza, DELETE elimina
- Siempre devolver schemas Out, nunca los modelos ORM directamente
- Validar negocio en el router, no en el modelo
- Generar asientos contables dentro de la misma transacción de base de datos

Frontend:

- Componentes: PascalCase (ProductCard.jsx)
- Rutas admin: /kebab-case (e.g. /contabilidad)
- Estado local con useState, nunca variables globales
- Errores siempre con toast.error(), éxitos con toast.success()
- Recargar datos después de cualquier mutación (create/update/delete)
- Colores de marca: #1E3A8A (azul), #FFD100 (amarillo) — no cambiar

Depuración común

Problema	Causa probable	Solución
Error CORS en el browser	Backend no incluye el origen del frontend en CORS de main.py	Agregar el URL del frontend en CORS de main.py
422 Unprocessable Entity	El JSON enviado no cumple el schema de validación	Revisar los docs para ver el schema esperado. Verificar tipos de datos.
Stock no se reduce	La venta no encontró la variante por el ID de venta	Asegurarse de enviar variant_id en el SaleItemCreate

Imagen no carga en tienda	URL relativa /uploads/... sin proxy correctamente	Verificar que backendext.config.js tiene el rewrite de /uploads a :8000
Categorías no aparecen	El endpoint /api/categories devolvió las categorías se auto-poblan al primer GET — reiniciar backend	
SQLite "disk I/O error"	Problema de permisos en la carpeta	Verificar que backend/ tiene permisos de escritura

Este documento cubre la versión 2.0.0 del sistema TodoTec desarrollada en Junio 2026. Para dudas o soporte de desarrollo contactar a Jose Rojas — joserojasuribe416@gmail.com